Putting It All Together: Building a Simple README Agent

Now that we understand all the components of our framework, let's see how they work together by building a simple but practical agent. We'll create an agent that can analyze Python files in a project and write a README file. This example will demonstrate how our modular design makes it straightforward to assemble an agent from well-defined components.

Understanding Our Agent's Purpose

Before we dive into the code, let's understand what we want our agent to do. Our README agent will:

1. Look for Python files in a project directory
2. Read the contents of each file
3. Analyze what it finds
4. Generate a README based on its analysis

This task is perfect for demonstrating our framework because it requires the agent to make decisions about which files to read, process information iteratively, and produce a final output.

Defining the Goals

Let's start by defining what our agent should achieve. We use goals to give the agent its purpose and guide its decision-making:

```
goals = [
    Goal(
        priority=1,
        name="Gather Information",
        description="Read each file in the project"
    ),
    Goal(
        priority=2,
        name="Terminate",
        description="Call the terminate call when you have read all the files."
                    "and provide the content of the README in the terminate message"
    )
]
```

Notice how we break the task into two clear goals. The first goal drives the agent to explore the project's files, while the second ensures it knows when to stop and produce output. We give both goals equal priority (priority=1) since they're sequential steps in the process.

Creating the Actions

Next, we define what our agent can do by creating its available actions. We need three basic capabilities:

```
def read_project_files(name: str) -> str:
    with open(name, "r") as f:
        return f.read()

def list_project_files() -> List[str]:
    return sorted([file for file in os.listdir(".")
                   if file.endswith(".py")])

# Together these actions span your descriptions
action_registry = ActionRegistry()
action_registry.register(Action(
    name="list_project_files",
    function=list_project_files,
    description="Lists all files in the project.",
    parameters=[],
    terminal=False
))

action_registry.register(Action(
    name="read_project_file",
    function=read_project_file,
    description="Reads a file from the project.",
    parameters=[
        {"type": "object",
         "properties": {
             "name": {"type": "string"}
         },
         "required": ["name"]
        }
    ],
    terminal=False
))

action_registry.register(Action(
    name="terminate",
    function=lambda message: f"{message}\nTerminating...",
    description="Terminates the session and prints the message to the user.",
    parameters=[
        {"type": "object",
         "properties": {
             "message": {"type": "string"}
         },
         "required": []
        }
    ],
    terminal=True
))
```

Each action is carefully designed with:

- A clear name that describes its purpose
- A function that implements the action
- A description that helps the LLM understand when to use it
- A schema defining its parameters
- A terminal flag indicating if it ends the agent's execution

Choosing the Agent Language

For our README agent, we'll use the function calling language implementation because it provides the most reliable way to structure the agent's actions:

```
agent_language = AgentFunctionCallingActionLanguage()
```

This choice means our agent will use the LLM's built-in function calling capabilities to select actions. The AgentLanguage will:

1. Format our goals as system messages
2. Present our actions as function definitions
3. Maintain conversation history in the memory
4. Parse function calls from the LLM's responses

Setting Up the Environment

Our environment is simple since we're just working with local files:

```
environment = Environment()
```

We use the default Environment implementation because our actions are straightforward file operations. For more complex agents, we might need to customize the environment to handle specific execution contexts or error cases.

Assembling and Running the Agent

Now we can bring all these components together:

```
# Create an agent instance with our components
agent = Agent(
    goals=goals,
    agent_language=AgentFunctionCallingActionLanguage(),
    action_registry=action_registry,
    generate_response=generate_response,
    environment=environment
)

# Run the agent with our task
user_input = "Write a README for this project."
final_memory = agent.run(user_input)
```

When we run this agent, several things happen:

1. The agent receives the user's request for a README
2. It uses list_project_files to discover what files exist
3. It uses read_project_file to examine each relevant file
4. When it has gathered enough information, it uses terminate to provide the README content

Understanding the Flow

Let's walk through a typical execution:

1. First Iteration:
   - Agent constructs prompt with goals and available actions
   - LLM decides to list files first (logical starting point)
   - Environment executes list_project_files
   - Memory stores the list of files
2. Middle Iterations:
   - Agent includes file list in context
   - LLM chooses files to read based on their names
   - Environment executes read_project_file for each chosen file
   - Memory accumulates file contents
3. Final Iteration:
   - Agent determines it has enough information
   - LLM generates README content
   - Agent uses terminate action to deliver the result

Making It Work

The modular design means we could easily modify this agent to:

- Handle different file types by adding new actions
- Generate different documentation by changing the goals
- Work with remote files by modifying the environment
- Use different LLM providers by changing the agent language

This example demonstrates how our framework's separation of concerns makes it easy to create focused, task-specific agents. Each component has a clear responsibility, making the code easy to understand and modify. The GAME architecture lets us think about each aspect of the agent's behavior independently while ensuring they work together seamlessly.

Remember, this is just a starting point. With this foundation, we can build more sophisticated agents by:

- Adding more complex actions
- Implementing smarter memory management
- Creating specialized environments
- Developing custom agent languages for specific needs

The key is that our framework makes these extensions possible without having to change the core agent loop or other components. This modularity is what makes our framework both powerful and practical.

The Complete Code for Our Agent

```
def main():
    # Define the agent's goals
    goals = [
        Goal(priority=1, name="Gather Information", description="Read each file in the project"),
        Goal(priority=2, name="Terminate", description="Call the terminate call when you have read all the files."
                                                       "and provide the content of the README in the terminate message")
    ]

    # Define the agent's language
    agent_language = AgentFunctionCallingActionLanguage()

    def read_project_files(name: str) -> str:
        with open(name, "r") as f:
            return f.read()

    def list_project_files() -> List[str]:
        return sorted([file for file in os.listdir(".") if file.endswith(".py")])

    # Define the action registry and register some actions
    action_registry = ActionRegistry()
    action_registry.register(Action(
        name="list_project_files",
        function=list_project_files,
        description="Lists all files in the project.",
        parameters=[],
        terminal=False
    ))
    action_registry.register(Action(
        name="read_project_file",
        function=read_project_file,
        description="Reads a file from the project.",
        parameters=[
            {"type": "object",
             "properties": {
                 "name": {"type": "string"}
             },
             "required": ["name"]
            }
        ],
        terminal=False
    ))
    action_registry.register(Action(
        name="terminate",
        function=lambda message: f"{message}\nTerminating...",
        description="Terminates the session and prints the message to the user.",
        parameters=[
            {"type": "object",
             "properties": {
                 "message": {"type": "string"}
             },
             "required": []
            }
        ],
        terminal=True
    ))

    # Define the environment
    environment = Environment()

    # Create an agent instance
    agent = Agent(goals, agent_language, action_registry, generate_response, environment)

    # Run the agent with user input
    user_input = "Write a README for this project."
    final_memory = agent.run(user_input)

    # Print the final memory
    print(final_memory.get_memories())


if __name__ == "__main__":
    main()
```